Bare-Metal OS for a Raspberry Pi4

Project Report for the Operating Systems Course
University of Basel, Spring Semester 2026

Authors: Andrea Seehuber
Luca Fässler

Contents

1. Introduction

The aim of this project was to implement a small bare-metal operating system for the Raspberry Pi4. The system is loaded directly as a kernel image and does not rely on Linux, a standard C runtime, processes, filesystems, or user-space services. Its main purpose is to demonstrate how core operating system mechanisms can be built from scratch on real hardware.

The project focusses on bootstrapping, kernel initialization, cooperative multitasking, context switching, synchronization, hardware drivers, and interactive system control. The resulting system is not intended to be a general-purpose operating system. Instead, it is a compact embedded kernel that can schedule multiple kernel tasks, communicate over UART, render text output to HDMI, access external hardware over I2C, and provide runtime diagnostics.

2. Hardware and Software Environment

The target platform is a Raspberry Pi4 Model B. The board is based on a Broadcom BCM2711 system-on-chip with a 64-bit ARM Cortex-A72 CPU. For this project, the Raspberry Pi is used without a conventional operating system. The firmware loads the custom kernel image directly from the boot partition, after which all further initialization is performed by the project code itself. As an extension module, the system uses the Raspberry Pi Sense HAT. This add-on board is connected through the 40-pin GPIO header and provides an 8x8 RGB LED matrix, a five-way joystick, and several environmental and motion sensors. In this project, the relevant Sense HAT components are the joystick, the LED matrix, and the sensors for pressure, humidity, and temperature. These devices are accessed through the Raspberry Pi's I2C controller.

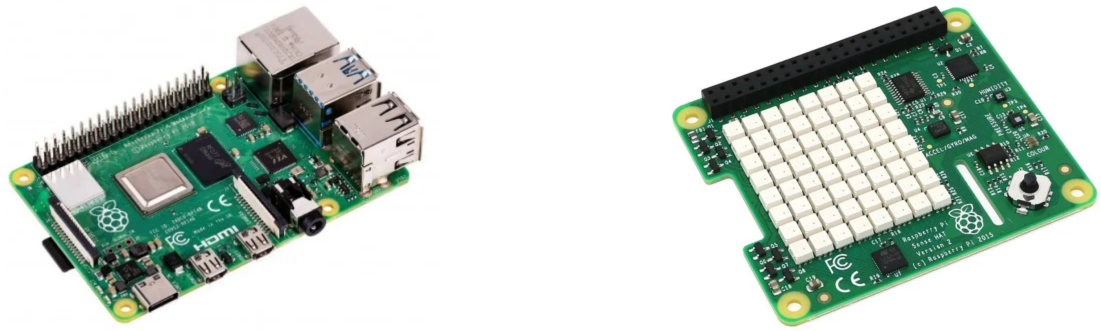

Figure 1: Raspberry Pi4 Model B and Raspberry Pi Sense HAT used as the hardware platform. Images adapted from Pi-Shop.ch[1].

UART is used as the primary debugging and control interface. Access from a development machine requires a USB-to-UART adapter and a serial terminal program such as PuTTY or minicon. HDMI is used as the main visual output device. Any standard HDMI display can be connected using a suitable micro-HDMI to HDMI cable or adapter.

The software is written mainly in C, with architecture-specific AArch64 assembly for early bootstrapping and context switching. The project is built using a Makefile. The build system uses an AArch64 bare-metal cross toolchain and compiles the code in freestanding mode, without the standard C library or startup files. The linker script defines the memory layout of the kernel image, and the final binary is converted into a Raspberry Pi compatible `kernel8.img`.

The implementation was developed with reference to the tutorial series *Writing a Bare-Metal Op-*

erating System for Raspberry Pi4. The tutorial provided the initial orientation for the bare-metal boot process, cross-compilation setup, linker configuration, and the general structure of an AArch64 Raspberry Pi kernel[2]. For hardware-specific register-level programming, the project also relies on the official *BCM2711 ARM Peripherals* documentation published by Raspberry Pi Ltd[3].

3. System Architecture

The system is based on a small monolithic bare-metal kernel, so all code runs in the same privileged address space. It is designed in a layered structure:

- At the lowest level, the kernel interacts with the Raspberry Pi4 peripherals through **memory-mapped I/O (MMIO)**. Peripheral registers are represented as fixed physical adresses derived from the BCM2711 peripheral base addresses[3]. Drivers such as UART, GPIO, I2C, the interrupt controller, the timer, and the HDMI mailbox interface read and write these registers through small MMIO helper functions.
- Above the MMIO layer, the kernel provides **hardware drivers and device abstractions**. UART provides the serial console and shell input. GPIO and the interrupt controller are used for the Sense HAT joystick interrupt. I2C provides communication with the Sense HAT devices, including the LED matrix controller, joystick state registers, and environmental sensors. HDMI output is initialized through the Raspberry Pi firmware mailbox interface, which returns a framebuffer that the kernel renders into directly.
- The middle layer consists of **kernel mechanisms**: task management, the cooperative scheduler, synchronization primitives, wait queues, logging, tracing, heap management, and diagnostics. The scheduler runs kernel tasks, mutexes protect shared resources, and the diagnostic code observe task state, stack usage, heap usage, and runtime activity.
- The upper layer consists of kernel tasks. **System tasks** are part of the infrastructure and are created during kernel initialization. They must remain alive because other components depend on them and are therefore protected against explicit stop requests. Examples are the idle task, shell-related infrastructure, joystick handling and LED rendering. **User-controllable tasks**, such as demo workers, dashboards and games, can be started and stopped through the UART shell or the joystick menu.

4. Core Components

4.1. Bootstrapping, MMIO, and Interrupt Setup

The kernel starts without support from a host OS or a C runtime. The first executed project code is AArch64 assembly. It sets up the initial stack, clears the `.bss` section, and then branches into the C kernel entry point. Clearing `.bss` is required because global kernel objects such task tables, mutexes, device state, and buffers must start in a known zero-initialized state.

After entering C code, the kernel initializes the hardware-facing subsystems before starting the scheduler. UART is initialized early to provide debug output. The interrupt controller is configured for the interrupts used by the kernel: the mini UART receive interrupt, the generic timer interrupt, the GPIO interrupt used by the Sense HAT joystick, and the I2C interrupt. The IRQ dispatcher acknowledges the interrupt at the GIC CPU interface, dispatches to the responsible subsystem, and then signals end-of-interrupt back to the controller.

4.2. Task System and Cooperative Scheduler

The task system is based on a statically allocated task table. Each **task control block (TCB)** contains the task ID, state, flags, saved stack pointer, entry function, wakeup tick, name and private stack. Dynamic allocation is deliberately avoided for TCBs and stacks, so the maximum number of tasks and the memory cost of each task are known at compile time. Task creation prepares the TCB and constructs an artificaial initial stack frame. This frame matches the layout expected by the AArch64 context switch routine. The saved return address points to `task_bootstrap()`. When the scheduler selects the task for the first time, the context switch restores this prepared context and returns into the bootstrap function. The same low-level context switch mechanism is therefore used both for starting a new task and for resuming an existing task.

The **scheduler** is cooperative. A task runs until it explicitly yields, sleeps, blocks, or exits. The scheduler scans the task table in round-robin order and selects the next ready non-idle task. If no ordinary task is ready, the idle-task is selected. The idle task executes a wait-for-event instruction and yields again, so the CU is not kept in a busy loop while no task can make progress. The task state encode why a task is or is not runnable:

- A `READY` task may be selected by the scheduler.
- A `RUNNING` task currently owns the CPU.
- A `SLEEPING` task is reactivated by the timer interrupt once a tick is reached.
- A `BLOCKED` task waits for an event, such as a mutex becoming available or a device interrupt.
- A `DYING` task is no longer scheduled and is cleaned up by the scheduler at a safe point.

Not all tasks are driven the same way. Some tasks are purely cooperative: they perform, work, sleep for a number of ticks, and then continue. Examples include periodic demo tasks, dashboards, and visualization. Their timing is based on explicit calls to `task_sleep()`.

Other tasks are interrupt-driven. They normally block and are only made ready when an interrupt indicates that work is available. The joystick task is the clearest example. The GPIO interrupt handler detects the Sense HAT joystick interrupt line, clears the GPIO event, and wakes the joystick task. The task then runs outside interrupt context, reads the joystick state over I2C, converts the hardware state into logical menu events, and dispatches them to the menu code. Sleeping tasks are also connected to interrupt handling. The generic timer interrupt increments the software tick counter and checks all sleeping tasks. If a task's wakeup tick has reached, the timer code changes its state back to `READY`. The timer interrupt does not preemptively switch tasks but only updates the scheduler-visible state.

4.3. Synchronization and Shared Hardware Access

Even with cooperative scheduling, **synchronization** is required because multiple tasks can acces shared kernel objects and because interrupts may update state asynchronously. The kernel therefore uses short critical sections where interrupts are disabled while sensitive data structures are modified. This is used for operations such as changing task state, waking tasks, updating scheduler data, or manipulating synchronization primitves.

For longer shared-resource protection, the kernel provides **mutexes**. A mutex records whether it is locked and which task owns it. If another task tries to acquire the mutex while it is already locked,

the task is put into a wait queue, marked as blocked, and the scheduler selects another runnable task. When the mutex is released, one waiting task is woken up and can continue later.

The most important protected shared resource is the I2C bus. Several independent components compete for the same bus:

- The environmental task reads pressure, humidity, and temperature sensors.
- The joystick task reads Sense HAT joystick state after GPIO interrupt.
- LED-related tasks update the Sense HAT LED matrix.
- Dashboard and shell commands may request recent sensor data or trigger display updates.

These operations must not interleave because an I2C transaction consists of multiple ordered register accesses. The global I2C bus mutex serializes these transactions. Every driver-level access to a Sense HAT device must acquire the bus lock before starting communication and release it afterwards. This meas the synchronization boundary is not the individual sensor or LED matrix, but the bus itself. Protecting the bus prevents one task from changing the I2C target address, FIFO state, or transfare control while another transaction is in progress.

A similar ownership principle is used for display output. HDMI output is divides into panes, and tasks can acquire exclusive ownership of a pane before rendering application-specific output. The LED matrix is also treated as a shared hardware resource. Instead of allowing arbitrary concurrent writes, LED output is routed through controlled rendering code so that hardware access remains centralized.

4.4. HDMI Rendering

HDMI output is one of the more complex parts of the project. The kernel requests a framebuffer from the Raspberry Pi firmware through the mailbox property interface. After initialization, rendering is performed directly by writing pixels into this framebuffer. Since the kernel does not use a graphical library or terminal emulator, text output, scrolling, clearing, and application views all have to be implemented manually.

The final design does not redraw the screen line by line on every update. Instead, HDMI output is modeled as a **character-cell display with backing state**. Each pane stores the character and color information for its cells. When code writes to a pane, it updates the cell state and marks only the affected cells as dirty. A later presentation step flushes dirty cells by converting their glyphs into framebuffer pixels. The dirty-cell model is important for responsiveness. A full redraw of the HDMI framebuffer is expensive and would let display output dominate CPU time and disturb the cooperative scheduler. Dirty-cell flushing limits the amount of work per update and makes it possible to budget rendering work across scheduler iterations.

4.5. Sense HAT LED Matrix Rendering

The Sense HAT LED matrix is also treated as shared output device. The matrix has 64 logical pixels, but the device expects color data to be transferred through the Sense HAT I2C interface. The kernel therefore builds LED frames in memory first and then sends the complete frame to the matrix controller. The LED matrix competes with the environmental sensors and joystick handling for the same I2C bus. Therefore, presenting a LED frame also has to respect the global I2C lock. This

is especially relevant because frame updates are larger than simple sensor register reads: sending a full LED frame means transferring many color bytes, so it must be treated as an exclusive bus operation.

4.6. Diagnostics

Because the system runs without Linux, there are no external tools such as `top`, `dmesg`, or a process monitor. The kernel therefore implements its own diagnostics. The shell can print scheduler state, task states, heap usage, stack high-water marks, logs, and trace entries. Since log entries are heap-allocated, heap diagnostics also show whether runtime logging remains bounded. The diagnostics dashboard renders selected runtime data live to the HDMI main pane.

5. Evaluation and Limitations

The project reached its main goal of booting a custom kernel on the Raspberry Pi4 and combining task scheduling, interrupt handling, UART control, HDMI output, Sense HAT support, and runtime diagnostics in one system. The result is usable for demonstration and experimentation, especially because tasks can be started and stopped interactively and the system can show its internal state through the shell and HDMI dashboards.

However, several parts of the implementation remain fragile. The I2C subsystem is the most important example. Although access to the bus is serialized with a global lock, communication with the Sense HAT devices is still timing-sensitive and occasionally unreliable. This affects joystick input, sensor reads, and LED matrix updates, because all of them share the same physical bus. The locking mechanism prevents obvious concurrent transactions, but it does not fully eliminate lower-level communication problems such as missed device responses, incomplete transfers, or recovery after a failed transaction.

The HDMI implementation also required several iterations. A simple line-based redraw approach was too expensive and interfered with the cooperative scheduler. The final dirty-cell model improved this by flushing only changed character cells, but it also made the display code more complex. Correct pane ownership, cursor handling, scrolling, and partial redraws are therefore still more error-prone than simple serial output over UART.

The main architectural limitation is the cooperative scheduler. It is compact and understandable, but it depends on every task yielding, sleeping, or blocking regularly. A task that performs long-running work without yielding can reduce system responsiveness. In addition, all code runs in one privileged address space, so there is no user/kernel separation, virtual memory, process isolation, filesystem support, or memory protection.

Future work should therefore focus less on adding more demo tasks and more on robustness. The I2C layer would benefit from better error handling, timeouts, bus recovery, and clearer transaction-state tracking. A preemptive scheduler would make the system more responsive under faulty or CPU-heavy tasks. Stronger memory and stack checks would make task execution safer, and persistent storage could be added for logs, configuration, or collected sensor data.

A. Appendix - User Manual

This appendix describes how to operate the bare-metal Raspberry Pi OS after it has been built and booted on the Raspberry Pi 4.

A.1. System Startup

The system is built into a Raspberry Pi kernel image named `kernel8.img`. After copying the image to the Raspberry Pi boot medium together with the required firmware files, the system starts automatically when the Raspberry Pi is powered on.

During boot, the kernel initializes UART, HDMI output, the heap, the scheduler, interrupts, the timer, I2C, and the Sense HAT device. After initialization, the following system tasks are started automatically:

- `shell`: provides the UART command interface.
- `hdmi_console`: mirrors console output to the HDMI main pane when it is not occupied by another task.
- `joystick`: reads Sense HAT joystick input and controls the HDMI menu.
- `led`: owns the physical Sense HAT LED matrix and renders submitted LED frames.

Once the message `"Kernel initialized"` appears, the system is ready for use.

A.2. Main Control Interfaces

The system can be controlled in two ways:

- The UART shell: Commands are typed into a serial terminal connected to the Raspberry Pi UART. Pressing ENTER executes a command.The first interface is the UART shell. Commands are typed into a serial terminal connected to the Raspberry Pi UART. Pressing ENTER executes a command.
- The Sense HAT joystick menu: The menu is shown on the HDMI menu pane. It allows the user to start and stop tasks, inspect logs, show task information, and execute selected shell commands without typing over UART.

A.3. Shell Commands

The UART shell supports the following commands:

- `help`: Prints the list of available shell commands.
- `ps`: Prints all currently existing tasks with their task ID, state, and name.
- `startable`: Lists all tasks that can be started manually.
- `start <name>`: Starts a task by name. For example, `start gol` starts the Game of Life task.
- `stop <id|name>`: Requests a task to stop. The argument can either be a numeric task ID or a task name. For example, `stop gol` or `stop 5` stops the Game of Life task if it is running.
- `log <id|name>`: Prints the log entries of the selected task. The argument can either be a numeric task ID or a task name. For example, `log gol` or `log 5` prints the log of the Game of Life task if it is running.

- `log clear <id|name>`: Clears the log entries of the selected task. The argument can either be a numeric task ID or a task name. For example, `log clear gol` or `log clear 5` clears the log of the Game of Life task if it is running.
- `diag`: Prints a diagnostic snapshot containing uptime, CPU busy percentage, heap usage, heap block statistics, task stack usage, and scheduler counters.
- `heap dump`: Prints the current heap block layout.
- `trace dump`: Prints scheduler trace events such as context switches, sleeps, wakes, stops, and exits. Reading the trace consumes the current trace buffer.
- `trace clear`: Clears the scheduler trace buffer.
- `env`: Prints the latest environment sensor sample. This requires the `env` task to be running and to have produced at least one valid sample.
- `env history`: Prints recent environment samples from the history buffer. This also requires the `env` task to be running.
- `shutdown`: Performs a controlled kernel shutdown. The system releases display resources, clears the LED matrix, clears HDMI panes, disables interrupts, and parks the CPU.

A.4. Available User Tasks

- `heart`: A simple heartbeat task that periodically logs heartbeat messages.
- `fast`: A fast demo worker task that performs frequent work and yields often.
- `slow`: A slower demo worker task that performs longer work sections and yields less often than `fast`.
- `burst`: A demo task that periodically creates bursts of log activity and then sleeps.
- `gol`: Runs Conway's Game of Life on the Sense HAT LED matrix.
- `tictactoe`: Runs a Tic-Tac-Toe game using the Sense HAT joystick for input and HDMI/LED matrix for output.
- `env`: Reads environment data from the Sense HAT sensors and maintains a history buffer.
- `envled`: Displays a compact environment status visualization on the Sense HAT LED matrix based on the latest sensor data from `env`.
- `envdash`: Shows a live HDMI environment dashboard with temperature, humidity, pressure, uptime, and a legend explaining the LED matrix colors. This task depends on `env` for sensor data.
- `diagdash`: Shows a live HDMI diagnostics dashboard with uptime, CPU load, heap usage, and per-task stack usage.

A.5. Joystick Menu Operation

The HDMI screen is split into a main pane and a menu pane. The menu pane is controlled with the Sense HAT joystick:

- Long CENTER: Opens or closes the menu. Inside a submenu, a long Center press returns to

the main menu.

- UP and DOWN: Move the current selection.
- Short CENTER in the main menu: Executes the selected command or opens a submenu.
- Short CENTER in the log menu: Prints the selected task log.
- LEFT in the task menu: Starts the selected task.
- RIGHT in the task menu: Stops the selected task.

The joystick menu exposes common commands such as `ps`, `env history`, `trace dump`, `shutdown`, log viewing, and task start/stop operations.

A.6. Tic-Tac-Toe Controls

When the Tic-Tac-Toe task is running, the joystick is temporarily used by the game instead of the system menu.

In the mode menu, UP and DOWN select between Easy CPU, Hard CPU, and Two Players. CENTER starts the selected mode.

During the game:

- UP, DOWN, LEFT, and RIGHT move the cursor across the board.
- CENTER places the current player's mark.
- X always starts.

After the game ends:

- UP and DOWN select between *Play again*, *Change mode*, and *Exit game*.
- CENTER confirms the selected option.

A.7. System Shutdown

Individual tasks should be stopped with the `stop` command or through the joystick task menu. This allows the kernel to release resources such as the LED matrix or HDMI main pane.

The complete system is ended with:

```
shutdown
```

After shutdown, the kernel clears the displays, disables interrupts, and enters a halted wait loop. To restart the system, the Raspberry Pi must be reset or power-cycled.

References

[1] Pi-Shop.ch. *Pi-Shop.ch*. Swiss retailer used as the source for hardware component images. URL: https://www.pi-shop.ch/ (visited on 06/02/2026).

[2] Sypstraw. *Writing a "bare metal" operating system for Raspberry Pi 4*. Tutorial series for bare-metal Raspberry Pi 4 operating-system development. 2020. URL: https://www.rpi4os.com/ (visited on 05/31/2026).

[3] Raspberry Pi Ltd. *BCM2711 ARM Peripherals*. Official Raspberry Pi 4 BCM2711 peripheral documentation. Jan. 18, 2022. URL: https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008248-DS-1-bcm2711-peripherals.pdf (visited on 05/31/2026).